

```
State<FTCalculatorPage> {
  • @override
  • Widget build(BuildContext context) {
  •   return new Scaffold(
  •     appBar: new AppBar(
  •       title: new Text(widget.title),
  •     ),
  •     body: new Container()
  •   );
  • }
  • }
```

This `_FTCalculatorPageState` class extends the `State` class. In this class, you will create the rest of the code for your app. This has a `build` function, which returns a `Scaffold` object. The responsibility of the `build` function is to define the display structure of the widgets. `Scaffold` is a widget that contains basic UI widgets like app bar, floating action buttons, body, menus, and many more for the home screen. While creating the `AppBar` object, it accepts the title of the widget as a parameter. Therefore, when you run the app, it will display the title on the app bar. The `body` property of the `Scaffold` object holds the `Container` object, which represents the `Container` widget. The `Container` widget can have child widgets. It has some styling properties as well. If the container does not have any child widgets, it occupies the entire area on the screen.

7. Save and run the app. After a few moments, you will see the app running with a blank screen and having a title, FT Calculator. We suggest you modify the code to change the app color and the title of the app and run once again to see the changes you have made.

Now, you will need to come to your `Container` and start building the UI of the Calculator app.

8. You will need a number of button widgets to make your Calculator app. To create a custom button widget for your app, add a function named `ftCalcButton` by using the following code snippet:

```
Widget ftCalcButton(String buttonText) {
  • return new Expanded(
  •   child: new OutlineButton(
  •     padding: new EdgeInsets.all(24.0),
```